

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: K SUHAS VARMA

Roll No.: A24126552091

Date: 29-08-2026

TASK 3 — JAVASCRIPT, DOM MANIPULATION AND STUDENT PROFILE

1. TITLE

Develop a Dynamic Student Profile Application Using JavaScript and DOM Manipulation

2. AIM

To develop a dynamic Student Profile Application using JavaScript that creates a Student object using user-provided values and dynamically displays the student details on a web page using DOM manipulation.

3. OBJECTIVE

The objectives of this task are:

- To understand JavaScript classes and objects.
 - To create a Student class with appropriate properties.
 - To initialize student details using a constructor.
 - To accept student information from the user.
 - To create a Student object using the entered values.
 - To understand JavaScript event handling.
 - To use DOM methods to access HTML elements.
 - To dynamically create HTML elements using JavaScript.
 - To display student details dynamically on the webpage.
 - To apply CSS styling to the dynamically generated student profile.
 - To perform basic form validation using JavaScript.
-

4. THEORY

JavaScript is a scripting language used to add dynamic behavior and interactivity to web pages. It can respond to user actions, manipulate HTML elements, validate input, and dynamically update webpage content.

In this application, a JavaScript class named Student is created. The class contains the following properties:

- Name
- Roll Number
- Department
- CGPA

A constructor is used to initialize these properties when a Student object is created.

The application also uses the DOM (Document Object Model) to access and modify HTML elements dynamically.

The basic working flow is:

User Input → Button Click → JavaScript Event → Student Object → DOM Manipulation → Student Profile Display

JavaScript Class

A class provides a blueprint for creating objects. The Student class is used to represent student information.

Constructor

The constructor initializes the properties of each Student object.

Event Listener

The `addEventListener()` method is used to detect when the Display Student Profile button is clicked.

DOM Manipulation

JavaScript DOM methods such as:

- `getElementById()`
- `createElement()`
- `appendChild()`

are used to access existing elements and dynamically create new elements.

Input Validation

Before creating the Student object, the program checks whether all required fields have been entered. If any field is empty, an alert message is displayed.

CSS Styling

CSS is used to style the main container, input fields, button, and dynamically generated student profile.

5. ALGORITHM / PROCEDURE

1. Create an HTML file (index.html) containing the student input form.
2. Create labelled input fields for Name, Roll Number, Department, and CGPA inside a `<form>`.
3. Add a Create Profile button of `type="button"` so the form does not submit or reload the page.
4. Link an external CSS file (styles.css) to style the container, form, inputs, button, and profile card.
5. Create an external JavaScript file (script.js) and define a Student class.
6. Define the constructor with name, roll, department, and cgpa parameters and assign them to `this.name`, `this.roll`, `this.department`, and `this.cgpa`.
7. Add a `getDetails()` method to the Student class that returns an object containing Name, Roll No, Department, and CGPA.
8. Add a `DOMContentLoaded` event listener to ensure the script runs only after the page has loaded.
9. Access the Create Profile button and the profile container using `getElementById()`.
10. Add a click event listener to the Create Profile button.
11. Read and trim the values entered in the Name, Roll Number, Department, and CGPA fields.
12. Check whether any of the required fields are empty and display an alert("Please fill all fields.") if so.
13. Convert the entered CGPA to a number using `parseFloat()`, keeping the original text if it is not a valid number.
14. Create a new Student object using the entered values.

15. Clear any previously displayed profile using `innerHTML = ''`.
 16. Dynamically create a `<div>` with class "profile-card" and an `<h2>` heading using `createElement()`.
 17. Call `getDetails()` on the Student object and loop through the returned details.
 18. Dynamically create a `<dt>` and `<dd>` element for each detail and append them to a `<dl>` element.
 19. Append the heading and the `<dl>` list to the profile card, and append the profile card to the profile container.
 20. Save the HTML, CSS, and JavaScript files and open `index.html` in a web browser.
 21. Test the application with valid input and with one or more fields left empty.
 22. Verify the generated student profile against the expected details.
-

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="container">
    <h1>Student Profile</h1>
    <form id="student-form" class="form" onsubmit="return false;">
      <label> Name:
        <input type="text" id="name" placeholder="e.g. Ravi Kumar" required>
      </label>
      <label> Roll Number:
        <input type="text" id="roll" placeholder="e.g. A23126552001" required>
      </label>
      <label> Department:
        <input type="text" id="dept" placeholder="e.g. CSE (AI & ML)" required>
      </label>
      <label> CGPA:
        <input type="number" step="0.01" id="cgpa" placeholder="e.g. 8.42" required>
      </label>
      <button id="create-btn" type="button">Create Profile</button>
    </form>
    <div id="profile-container"></div>
  </div>
  <script src="script.js" defer></script>
</body>
</html>
```

styles.css

```
body {
  font-family: Arial, Helvetica, sans-serif;
  background: #f4f6f8;
  color: #222;
  margin: 0;
  padding: 24px;
}
.container {
  max-width: 640px;
  margin: 0 auto;
}
.form {
  display: flex;
  flex-direction: column;
```

```

        gap: 10px;
        margin-bottom: 16px;
    }
    .form label {
        display: flex;
        justify-content: space-between;
        align-items: center;
        gap: 12px;
    }
    .form input {
        flex: 1;
        padding: 8px;
        border: 1px solid #cfd8dc;
        border-radius: 4px;
    }
    #create-btn {
        padding: 10px 14px;
        border: none;
        background: #0078d4;
        color: #fff;
        border-radius: 6px;
        cursor: pointer;
        width: 160px;
    }
    #create-btn:hover {
        background: #005a9e;
    }
    .profile-card {
        background: #fff;
        padding: 16px;
        border-radius: 8px;
        box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    }
    .profile-card h2 {
        margin-top: 0;
    }
    .profile-card dl {
        display: grid;
        grid-template-columns: 150px 1fr;
        column-gap: 12px;
        row-gap: 8px;
    }
    .profile-card dt {
        font-weight: 700;
        color: #444;
    }
    .profile-card dd {
        margin: 0;
    }
}

```

script.js

```

class Student {
    constructor(name, roll, department, cgpa) {
        this.name = name;
        this.roll = roll;
        this.department = department;
        this.cgpa = cgpa;
    }
    getDetails() {
        return {
            Name: this.name,
            'Roll No': this.roll,
            Department: this.department,
            CGPA: this.cgpa,
        };
    }
}

```

```

document.addEventListener('DOMContentLoaded', () => {
  const btn = document.getElementById('create-btn');
  const container = document.getElementById('profile-container');
  function createProfileCard(student) {
    container.innerHTML = '';
    const card = document.createElement('div');
    card.className = 'profile-card';
    const title = document.createElement('h2');
    title.textContent = 'Student Profile';
    card.appendChild(title);
    const dl = document.createElement('dl');
    const details = student.getDetails();
    for (const key in details) {
      const dt = document.createElement('dt');
      dt.textContent = key + ':';
      const dd = document.createElement('dd');
      dd.textContent = details[key];
      dl.appendChild(dt);
      dl.appendChild(dd);
    }
    card.appendChild(dl);
    container.appendChild(card);
  }
  btn.addEventListener('click', () => {
    const name = document.getElementById('name').value.trim();
    const roll = document.getElementById('roll').value.trim();
    const dept = document.getElementById('dept').value.trim();
    const cgpaVal = document.getElementById('cgpa').value.trim();
    if (!name || !roll || !dept || !cgpaVal) {
      alert('Please fill all fields.');
```

7. CODE EXPLANATION

Code	Explanation
class Student	Defines a class named Student that represents a student's details.
constructor(name, roll, department, cgpa)	Initializes the properties of a Student object when it is created.
this.name / this.roll / this.department / this.cgpa	Store the student's name, roll number, department, and CGPA on the object.
getDetails()	A method that returns an object containing the student's Name, Roll No, Department, and CGPA as key-value pairs.
document.addEventListener('DOMC ontentLoaded', ...)	Ensures the script runs only after the HTML document has fully loaded.
getElementById()	Selects an HTML element using its ID, such as the button and the profile container.
addEventListener('click', ...)	Detects and responds to the Create Profile button being clicked.
.value.trim()	Retrieves the text entered in an input field and removes extra leading/trailing spaces.
alert()	Displays a message to the user, used here for the "Please fill all fields."

Code	Explanation
	validation message.
parseFloat()	Converts the entered CGPA text into a floating-point number.
isNaN()	Checks whether the CGPA conversion failed, keeping the original text in that case.
new Student()	Creates a new object of the Student class using the entered values.
innerHTML = "	Clears any previously generated student profile before displaying a new one.
createElement()	Dynamically creates new HTML elements such as the profile card, heading, and list.
className	Assigns the "profile-card" CSS class to the dynamically created card element.
textContent	Sets the text content of the dynamically created heading, <dt>, and <dd> elements.
appendChild()	Adds a dynamically created element as a child of another element.
<dl>, <dt>, <dd>	Used to display the student details as a definition list of labels and values.
.profile-card	CSS class used to style the dynamically generated student profile card.
#create-btn:hover	Applies a darker background colour when the mouse pointer is over the Create Profile button.

8. TEST CASES AND EXECUTION DATA

Valid Input Test Cases

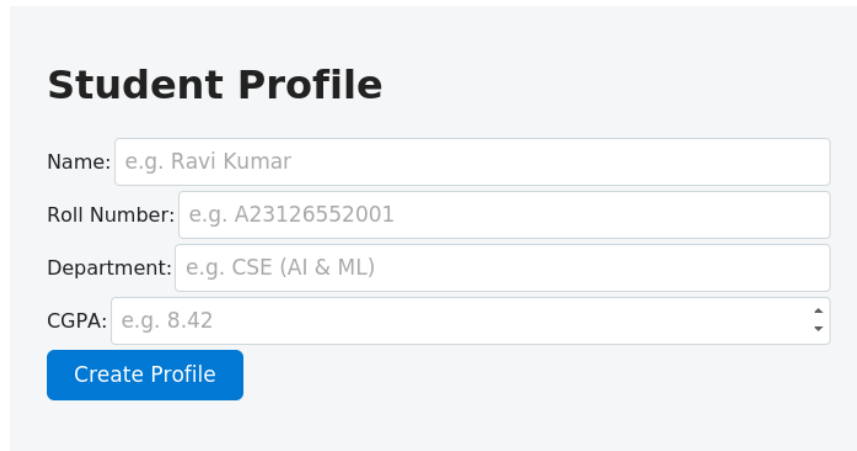
Test Case	Name	Roll No.	Department	CG PA	Expected Result	Actual Result	Status
TC-01	suhas varma	A24126552091	CSE	8.7	Student profile should be displayed	Student profile displayed correctly	Pass
TC-02	Priya	A23126552002	CSE	9.10	Student profile should be displayed	Student profile displayed correctly	Pass
TC-03	Arjun	A23126552003	ECE	7.85	Student profile should be displayed	Student profile displayed correctly	Pass

Validation Test Case

Test Case	Input	Expected Result	Actual Result	Status
TC-04	One or more fields left empty	Alert should display "Please fill all fields."	Alert displayed correctly	Pass

9. OUTPUT

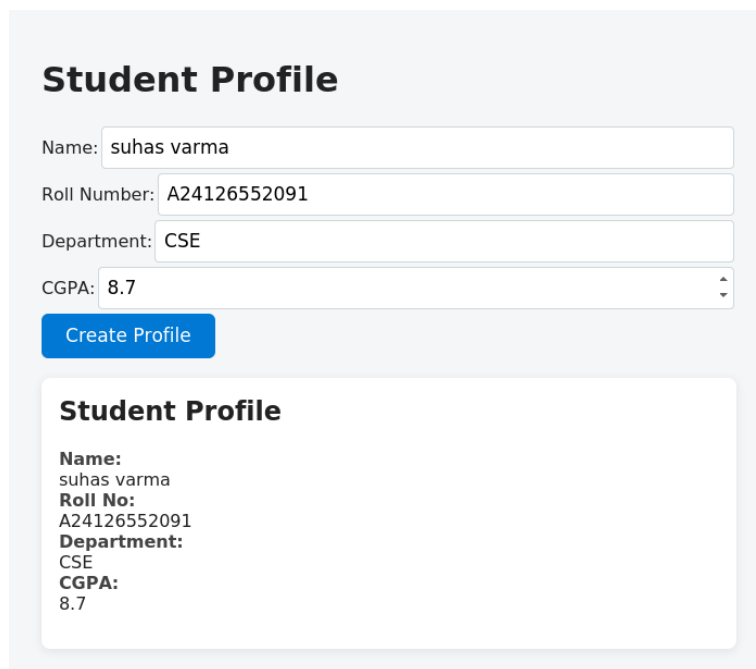
Output 1: Student Information Input Page



The image shows a web form titled "Student Profile". It contains four input fields: "Name:" with the placeholder text "e.g. Ravi Kumar", "Roll Number:" with the placeholder text "e.g. A23126552001", "Department:" with the placeholder text "e.g. CSE (AI & ML)", and "CGPA:" with the placeholder text "e.g. 8.42". Below these fields is a blue button labeled "Create Profile".

Figure 1: Student Profile application before displaying the student details.

Output 2: Dynamically Generated Student Profile



The image shows the same "Student Profile" form as in Figure 1, but now it displays the entered data. The input fields contain: "Name: suhas varma", "Roll Number: A24126552091", "Department: CSE", and "CGPA: 8.7". Below the "Create Profile" button, there is a new section titled "Student Profile" which displays the following details: "Name: suhas varma", "Roll No: A24126552091", "Department: CSE", and "CGPA: 8.7".

Figure 2: Student Profile dynamically generated using JavaScript DOM manipulation.

10. INTERVIEW QUESTIONS

Q1. What is a JavaScript class?

Answer: A JavaScript class is a blueprint for creating objects. It defines the properties and methods that objects created from the class can have.

Q2. What is the purpose of a constructor in a JavaScript class?

Answer: A constructor is a special method that is automatically called when an object is created. It is used to initialize the properties of the object.

Q3. What is DOM manipulation?

Answer: DOM manipulation is the process of using JavaScript to access, create, modify, or remove HTML elements dynamically in a web page.

Q4. What is the purpose of createElement()?

Answer: The createElement() method creates a new HTML element dynamically using JavaScript. In this application, it is used to create the student profile and its contents.

Q5. What is the purpose of appendChild()?

Answer: The appendChild() method adds a newly created HTML element as a child of an existing element.

Q6. Why is addEventListener() used in this application?

Answer: addEventListener() is used to detect the button click event and execute the code that creates and displays the student profile.

Q7. What happens if one of the input fields is empty?

Answer: The program checks whether any required field is empty. If a field is empty, an alert message "Please enter all student details." is displayed and the Student object is not created.

11. RESULT

Thus, a Dynamic Student Profile Application was successfully developed using a JavaScript class, a constructor, a getDetails() method, event handling, and DOM manipulation. Student details were accepted from the user, a Student object was created dynamically, and the student profile was generated as a definition list (<dl>, <dt>, <dd>) and displayed on the webpage with CSS styling. The application was successfully tested using valid inputs and with empty fields to verify the alert-based validation.